Fachhochschule Nordwestschweiz
Hochschule für Technik

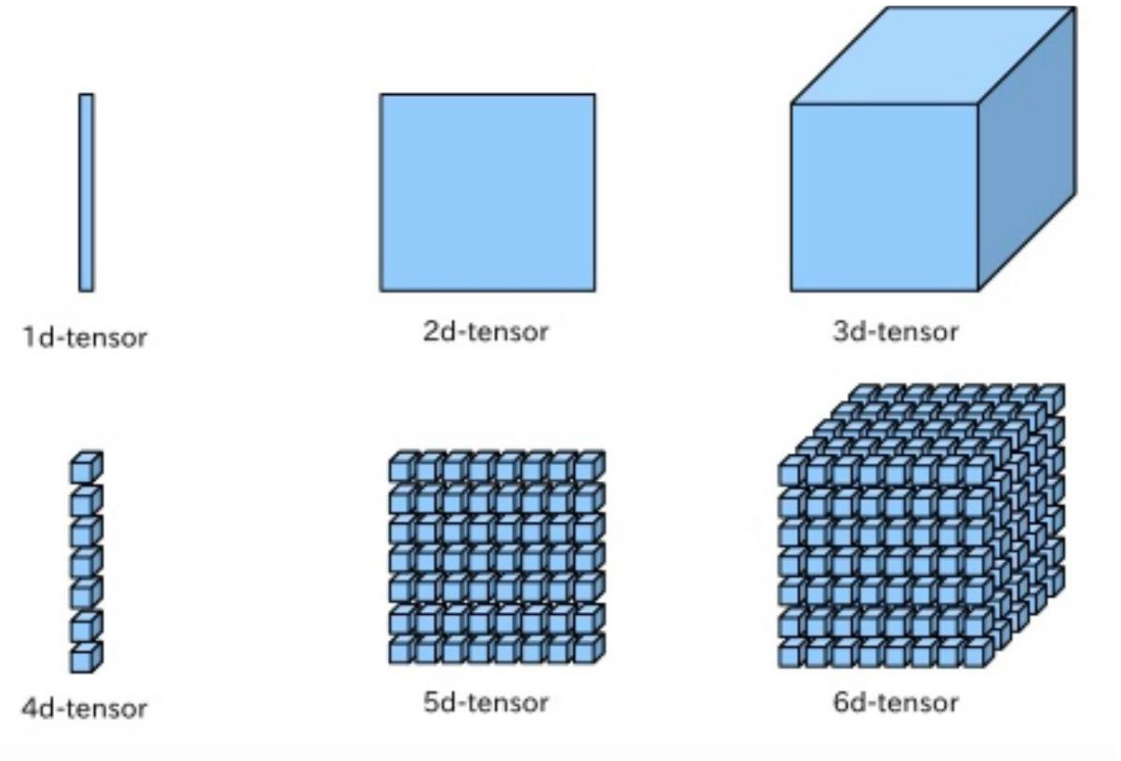
Intro PyTorch

Deep Learning

HS 2025

Martin Melchior

First Steps in PyTorch



Setup PyTorch and Jupyter

test in jupyter notebook

1. Create and activate new environment:

```
conda create --name del_hs25 python=3.9
conda activate del_hs25
```

3. Install jupyter lab, matplotlib, pandas:

```
conda install jupyterlab matplotlib pandas
```

2. Install PyTorch

NOTE: Latest PyTorch requires Python 3.9 or later.

PyTorch Build	Stable (2.8.0)			Preview (Nightly)	
Your OS	Linux	Mac		Windows	
Package	Pip	LibTorch		Source	
Language	Python			C++ / Java	
Compute Platform	~~CUDA 12.6~~	~~CUDA 12.8~~	~~CUDA 12.9~~	~~ROCm 6.4~~	Default
Run this Command:	`pip3 install torch torchvision`				

```
[1]: import torch

[2]: print (torch.__version__)
     2.8.0

[3]: print(torch.cuda.is_available())
     print(torch.backends.mps.is_available())
     False
     True
```

linux, windows

mac

PyTorch Tensors

- **Tensor** is a generalized matrix, a finite table of numerical values indexed along several discrete dimensions.
  - 0d tensor: a scalar.
  - 1d tensor: a vector (e.g. a sound sample).
  - 2d tensor: a matrix (e.g. a grayscale image).
  - 3d tensor: a vector of identically sized matrices (e.g. a RGB image).
  - 4d tensor: a matrix of identically sized matrices, or a sequence of 3d tensors (e.g. a sequence of RGB images, a video, dataset of images)

- Constrained structure of a tensor allows to be processed by using CPUs and GPUs, at [near] peak performance.
  - E.g. standard linear algebra operations, but also convolution, pooling, etc.

PyTorch Tensors

- Efficient tensor operations on CPU/GPU:
  - Very similar to numpy operations, incl broadcasting
  - standard linear algebra
  - deep-learning specific operations (conv, pooling)
- Automatic on-the-fly differentiation (autograd)
- Properties:
  - device: Tensors live on a device
  - dtype: by default `float32`
- Data I/O

Pytorch Cheat Sheet

Source: Francois Fleuret

Creation

- torch.empty(*size, ...)
- torch.zeros(*size, ...)
- torch.full(size, value, ...)
- torch.tensor(sequence, ...)
- torch.eye(n, ...)
- torch.from_numpy(ndarray)

Indexing, Slicing, Joining, Mutating

- torch.Tensor.view(*size)
- torch.cat(inputs, dimension=0)
- torch.chunk(tensor, nb_chunks, dim=0)[source]
- torch.split(tensor, split_size, dim=0)[source]
- torch.index_select(input, dim, index, out=None)
- torch.t(input, out=None)
- torch.transpose(input, dim0, dim1, out=None)

Filling

- Tensor.fill_(value)
- torch.bernoulli_(proba)
- torch.normal_([mu, [std]])

Pointwise math

- torch.abs(input, out=None)
- torch.add()
- torch.cos(input, out=None)
- torch.sigmoid(input, out=None)

Math reduction

- torch.dist(input, other, p=2, out=None)
- torch.mean()
- torch.norm()
- torch.std()
- torch.sum()

BLAS and LAPACK Operations

- torch.eig(a, eigenvectors=False, out=None)
- torch.lstsq(B, A, out=None)
- torch.inverse(input, out=None)
- torch.mm(mat1, mat2, out=None)
- torch.mv(mat, vec, out=None)

```
[1]: import torch
```

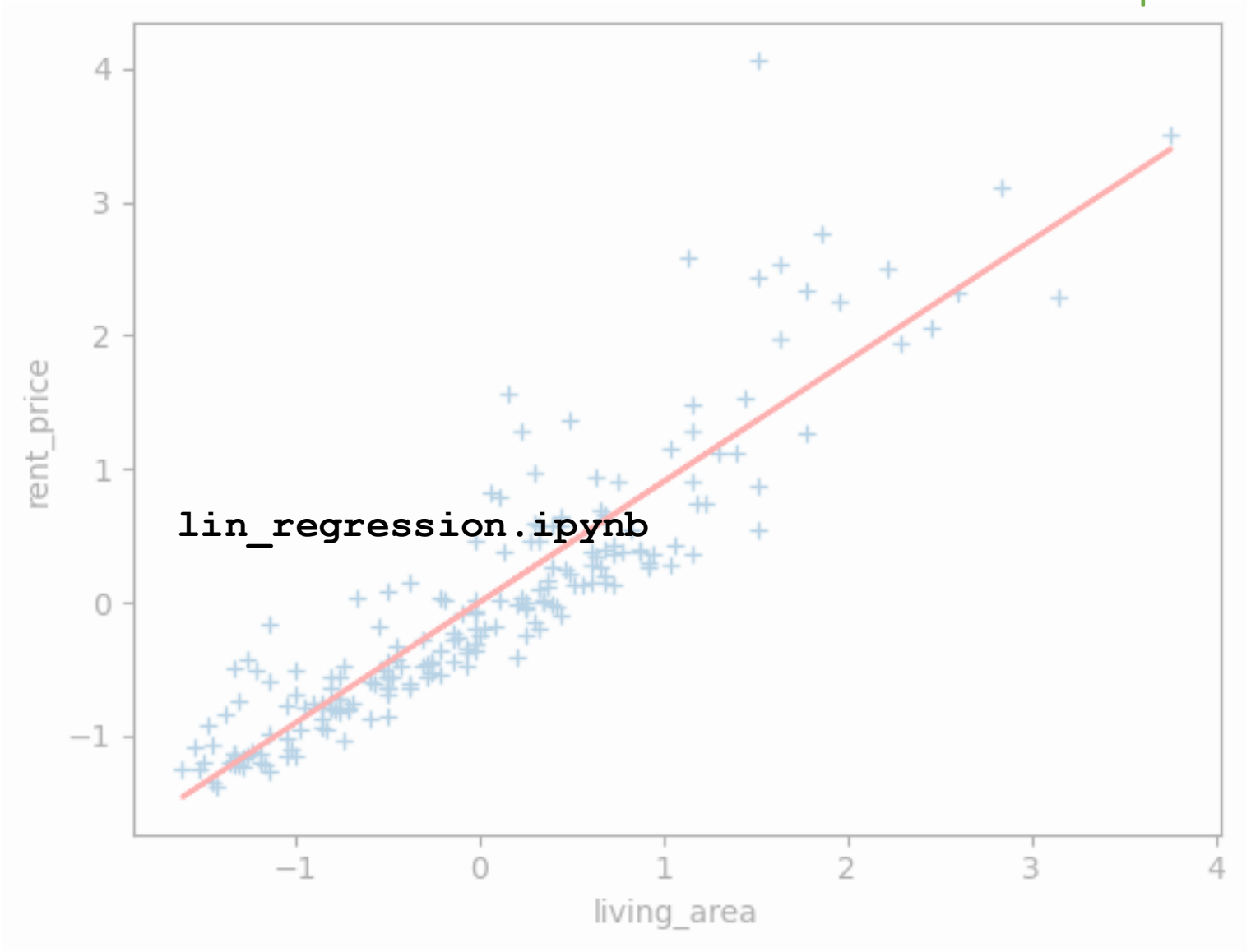
Initialization

```
[2]: # from lists
     x = torch.Tensor([5, 3])
     x, x.shape, x.dtype, x.device

[2]: (tensor([5., 3.]), torch.Size([2]), torch.float32, device(type='cpu'))

[3]: # with zeros or ones
     x = torch.zeros([3, 2])
     x, x.shape

[3]: (tensor([[0., 0.],
              [0., 0.],
              [0., 0.]]),
      torch.Size([3, 2]))

[4]: y = torch.ones([2, 3])
     y, y.shape

[4]: (tensor([[1., 1., 1.],
              [1., 1., 1.]]),
```

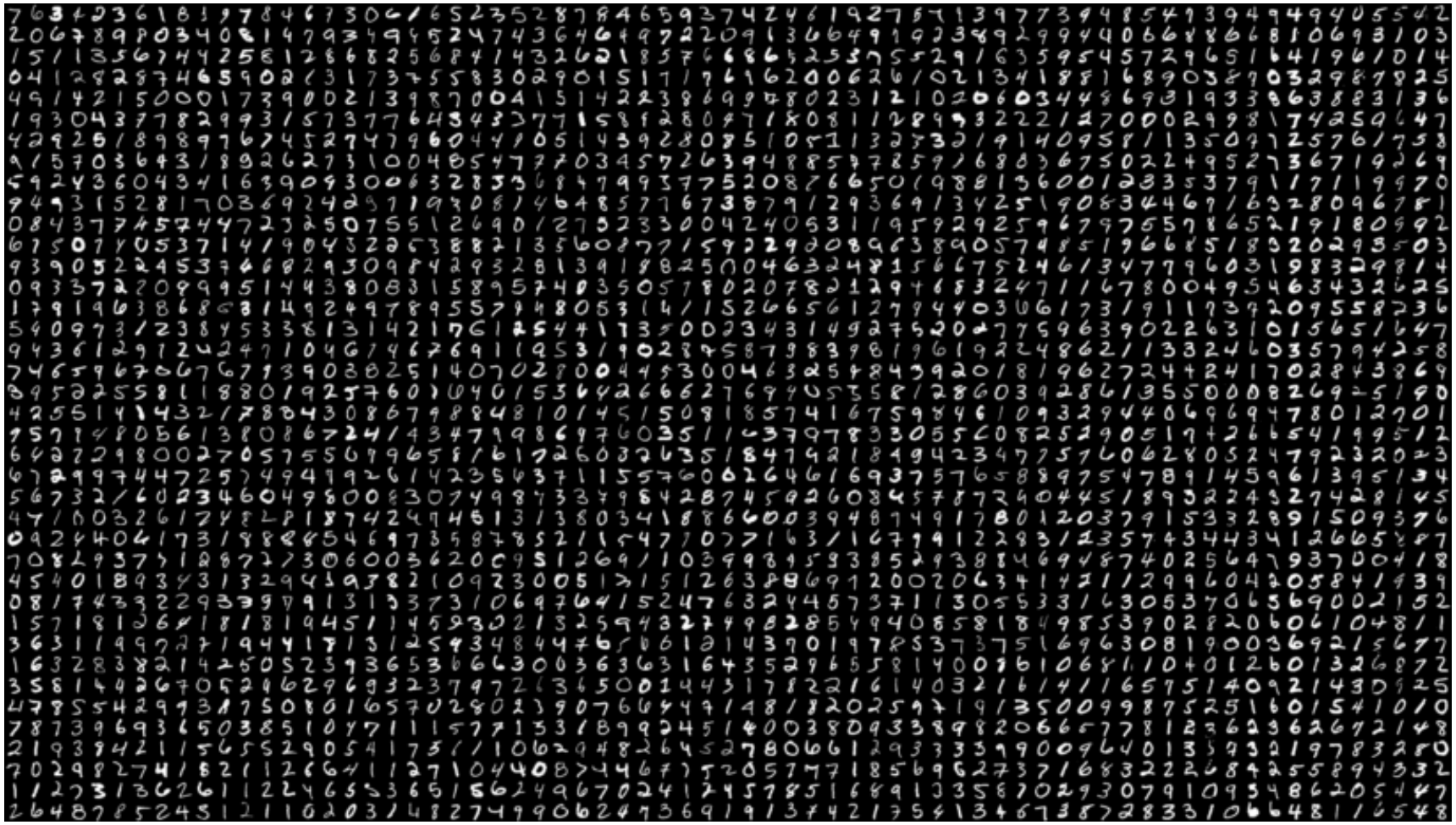
Hands-On `torch_tensor.ipynb`

Linear Regression

Hands-On



Loading Data, MNIST



Data Loading in PyTorch

Files, RAM, DB, …

class inherited from torch.utils.data.Dataset

- indexed (random) access to the samples e.g. (x, y)-pairs
- configure pre-processing steps (i.e. normalisation), etc.

see blog for custom datasets

torch.utils.data.DataLoader

- iterator over batches of samples: batch of x's, batch of y's
- shuffling of samples when iterator is reset
- parallelised access
- memory pinning
- etc.

see blog article for some explanation and additional recommendations

MNIST Dataset with PyTorch

Use datasets.MNIST to load 60’000 images for training and 10’000 for testing

```
import torch
from torchvision import datasets
from torchvision.transforms import ToTensor
import matplotlib.pyplot as plt
```

```
training_data = datasets.MNIST(
    root="data",
    train=True,
    download=True,
    transform=ToTensor()
)
```

for data downloaded to a subfolder of your working directory.

`transform`
for preprocessing pipeline
`ToTensor`
Transforms PIL images or numpy-arrays to tensor.

```
print(training_data)
```

```
Dataset MNIST
    Number of datapoints: 60000
    Root location: data
    Split: Train
    StandardTransform
Transform: ToTensor()
```

see torch.utils.data for further details on datasets and loading data

Loading MNIST with PyTorch

Elements are (x,y)-tuples with

x : a pytorch tensor with the 2d grayscale pixel values

y : an int with the label

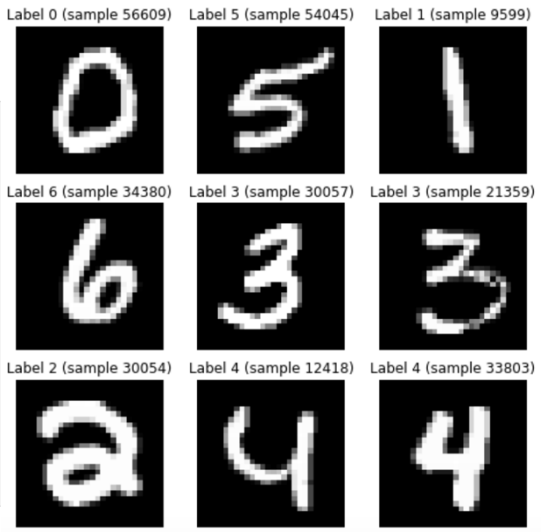
6th sample

```
print(len(training_data))
x, y = training_data[5]
print(type(x), x.shape, x.dtype)
print(type(y), y)
```

tensor with shape 1x28x28 with the first dimension corresponding to the color channel

```
60000
<class 'torch.Tensor'> torch.Size([1, 28, 28]) torch.float32
<class 'int'> 2
```

```
figure = plt.figure(figsize=(8, 8))
cols, rows = 3, 3
for i in range(1, cols * rows + 1):
    sample_idx = torch.randint(len(training_data), size=(1,)).item()
    img, label = training_data[sample_idx]
    figure.add_subplot(rows, cols, i)
    plt.title("Label %i (sample %i)"%(label,sample_idx))
    plt.axis("off")
    plt.imshow(img.squeeze(), cmap="gray")
plt.show()
```



DataLoader Class in PyTorch

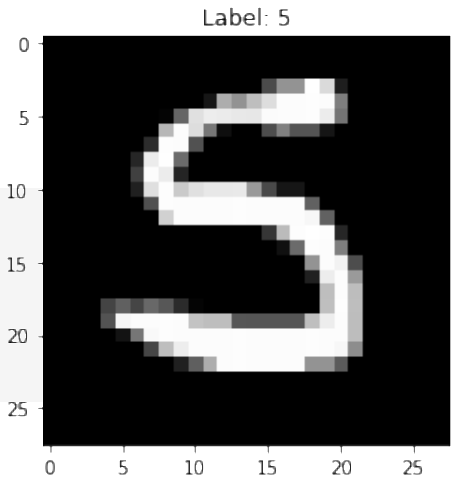
```
from torch.utils.data import DataLoader
train_dataloader = DataLoader(training_data, batch_size=64, shuffle=True)
```

```
data_train, labels_train = next(iter(train_dataloader))
print(f"Feature batch shape: {data_train.shape}")
print(f"Labels batch shape: {labels_train.shape}")
```

```
Feature batch shape: torch.Size([64, 1, 28, 28])
Labels batch shape: torch.Size([64])
```

tensor with shape (64x1x28x28) with the first dimension corresponding to the sample index in the batch

```
img = data_train[0].squeeze()
label = labels_train[0]
plt.imshow(img, cmap="gray")
plt.title(f"Label: {label}")
plt.show()
```



Hands-On

`load_mnist.ipynb`